Project 1: Rapid ETL Pipeline Development

Technical Documentation | DATA 430 - Data Engineering Practicum

1. Introduction

This proof-of-concept implements an automated Extract, Transform, Load (ETL) workflow for synthetic customer, product, and order data. Python Faker generates 1,000 records per entity, exported to CSV, JSON, and YAML: nine files representing 3,000 unique records. Apache Airflow validates and orchestrates processing; PostgreSQL stores the resulting relational data; Docker Compose manages the services.

The scope includes daily scheduling and manual execution, record-count and foreign-key checks, transaction-level exception handling, archival copies, and success/failure auditing.

2. System Architecture

Apache Airflow 2.10.5 uses a webserver for monitoring, a scheduler for task coordination, and LocalExecutor for running the DAG. PostgreSQL 16 contains separate databases: etl_project (business tables and ETL audit) and airflow_db (Airflow metadata). Docker mounts the host dags/, data/, scripts/ and sql/ directories for access inside containers.

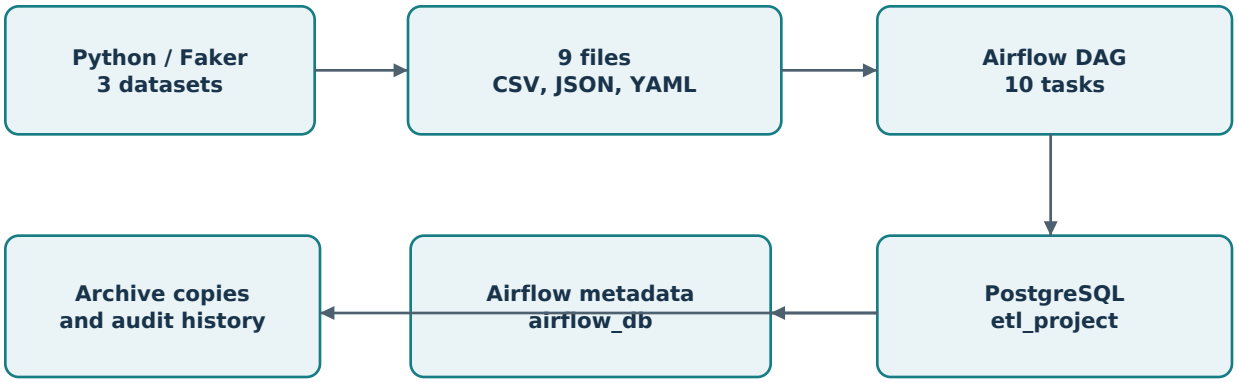


Figure 1. High-level data generation, orchestration, storage, and archive paths.

Component	Responsibility
Python / Faker	Create synthetic source data
pandas / PyYAML	Read and validate the three input formats
Airflow scheduler / webserver	Schedule, execute, monitor, and manually trigger tasks
PostgreSQL	Store relational data, metadata, and audit records
Docker Compose	Configure service containers, networking, and persistent storage

Configuration: Airflow is accessed at localhost:8080; PostgreSQL maps host port 5433 to container port 5432. Local credentials are kept in .env, excluded from Git; .env.example contains placeholders. A persistent Docker volume retains PostgreSQL data between restarts.

3. Database Design

The etl_project database uses PostgreSQL primary keys, uniqueness constraints, and a foreign key to preserve data integrity. Each business table also has an automatically populated created_at timestamp.

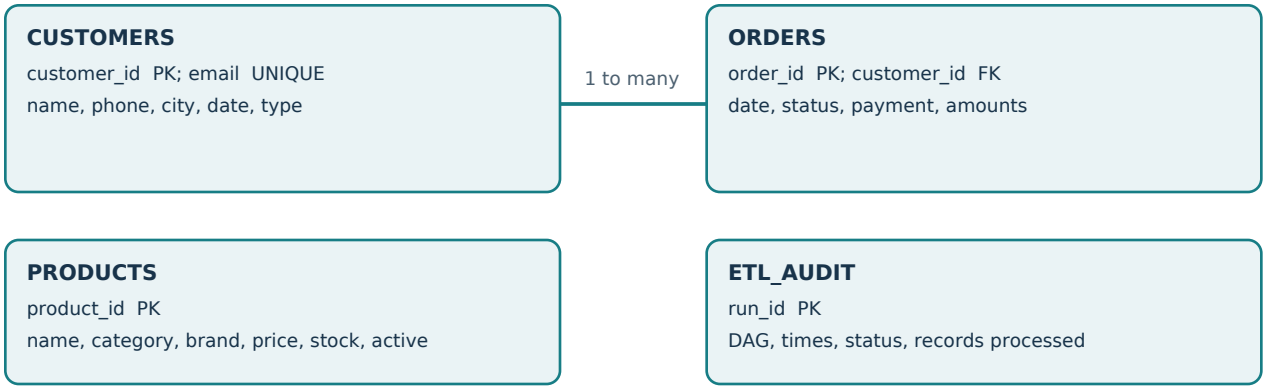


Figure 2. Customers and orders have a one-to-many relationship; products and audit records are independent.

Table	Key fields / principal attributes
customers	customer_id (PK); unique email; first_name, last_name, phone, city, registration_date, customer_type, created_at
products	product_id (PK); product_name, category, brand, unit_price DECIMAL(10,2), stock_quantity, is_active, created_at
orders	order_id (PK); customer_id (FK); order_date, order_status, payment_method, total_amount, discount_amount, shipping_cost, created_at
etl_audit	run_id (PK); dag_name, start_time, end_time, status, records_processed

Referential integrity

The orders.customer_id column references customers.customer_id; PostgreSQL rejects orders referencing customers that do not exist. Accordingly, customers are loaded before orders. The validation task also queries for unmatched customer references. The specified order schema contains no product identifier, so products are not directly linked to orders.

Audit metadata

The audit table stores a row for successful execution in the completion task and a failure entry from a DAG-level callback. In the current implementation, failure rows report zero processed records; successful timestamps are captured at completion rather than from the DAG run metadata.

4. ETL Data Flow

The DAG project1_etl_pipeline runs on @daily with catchup=False and can be manually triggered. Its ten tasks enforce the required validation, extraction, loading, verification, and completion order.

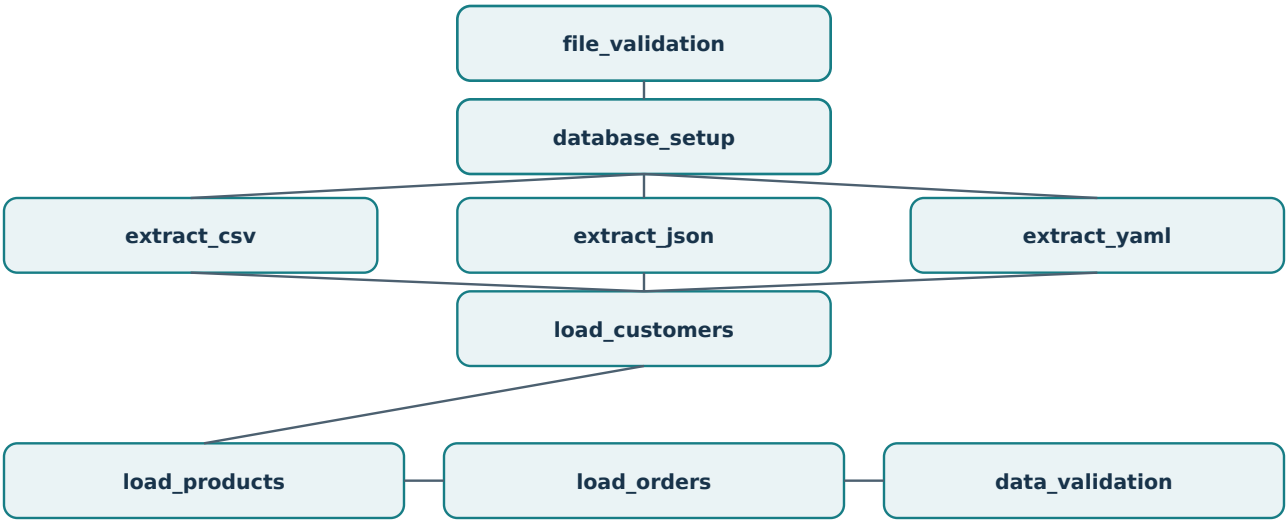


Figure 3. The three extraction tasks branch after database setup and converge before customer loading; completion follows data validation.

Stage / tasks	Processing performed
file_validation	Verify existence, readability, and 1,000-record count for each of nine files
database_setup	Execute schema.sql and clear the three business tables for full refresh
extract_csv / extract_json / extract_yaml	Read and validate respective customer, product, and order files in parallel
load_customers / load_products / load_orders	Insert 1,000 records into each table in dependency order
data_validation	Check the three row counts and unmatched customer IDs in orders
completion	Create timestamped archive copies of nine files and insert success audit row

Data handling

The three formats hold equivalent entity records, not 9,000 distinct records. All nine files are read during validation/extraction; CSV is read again by the loading functions for PostgreSQL insertion. Selecting a single canonical loading format prevents duplicate primary keys. The original source files remain available for subsequent daily runs.

5. Key Design Decisions

Python-based orchestration

Apache Airflow was chosen over NiFi for its Python-native DAG definition, explicit dependencies, integrated scheduling, and convenient task-level logging. LocalExecutor avoids additional distributed-worker services for this small individual project.

Containerized, reproducible runtime

Docker Compose packages PostgreSQL and the Airflow services in a consistent runtime. Volume mounts provide source files and DAG code to containers, while a named PostgreSQL volume preserves database contents. Configuration values are supplied through environment variables; secrets remain outside the public Git repository.

Multi-format extraction and canonical loading

The project demonstrates reading CSV, JSON, and YAML, but does not load all three copies of the same records. CSV supplies the insertion data, simplifying the proof of concept and avoiding duplicate customer, product, and order keys.

Ordered loading and validation

The extraction tasks can run concurrently after setup. Loading then proceeds sequentially: customers, products, orders. This satisfies the customer-to-order foreign-key dependency. Database validation executes after all inserts have completed.

Full-refresh strategy

The setup task truncates customers, products, and orders before reloading the full 1,000 records for each. This is straightforward and was verified through repeated runs without duplicate accumulation. It is appropriate for the project scale, but incremental loading/upserts would better suit larger or historical datasets.

Logging, auditing, and data retention

Airflow task logs and Python logging record processing and exceptions. Successful runs insert an audit row with 3,000 processed records; a DAG failure callback records unsuccessful runs. Completion makes timestamped archive copies instead of moving originals, allowing repeated execution.

Operational trade-offs

The pipeline commits loading tasks separately, so it does not provide a single atomic transaction spanning the full DAG. Failure after some inserts can temporarily leave partially reloaded business tables. Initial creation of the separate Airflow metadata database is also manual in the present configuration; a future iteration should automate first-time setup.

6. Error Handling and Data Validation

File-level controls

The first task confirms every required CSV, JSON, and YAML file is present, readable, and contains exactly 1,000 records. The three extraction tasks independently read their input formats and log record counts. Missing files, unreadable content, or incorrect counts raise exceptions so downstream processing does not continue.

Database-level controls

Loading functions check primary-key duplicates, and customer loading also checks email uniqueness. Each loading function uses psycopg2 transactions, commits on success, and rolls back its own transaction on failure. PostgreSQL primary-key, unique-email, and foreign-key constraints provide additional enforcement.

Workflow-level controls

The DAG retries failed tasks once after a one-minute delay. On final DAG failure, the registered callback writes a failed status to etl_audit. On successful completion, the last task writes a success audit entry and archives copies of the source files.

Validation checkpoints

When	Validation / action
Before setup	Nine files exist and are readable; each has exactly 1,000 records
During extraction	All three formats can be parsed and have expected counts
During loading	Duplicate key checks and database constraint enforcement
After loading	Each of customers/products/orders has exactly 1,000 rows
After loading	No orders reference missing customers
On completion / failure	Archive source copies and write success audit, or write failure audit

Limitations of these controls

Validation checks expected counts and the customer-order relationship but does not perform extensive semantic checks of prices, dates, allowed categorical values, or cross-format equality. Automated unit and integration test suites were not implemented; those are potential enhancements beyond the tested core workflow.

6. Testing Results and Recovery

Successful and repeat-run testing

The full ten-task DAG was manually triggered and completed successfully. PostgreSQL row counts were checked after execution and again following repeated runs, with no duplicate accumulation.

Business table	Expected	Observed	Result
customers	1,000	1,000	Passed
products	1,000	1,000	Passed
orders	1,000	1,000	Passed
TOTAL	3,000	3,000	Passed

Controlled missing-file failure

To test the pipeline failure path, customers.csv was temporarily renamed customers_test_backup.csv. The pipeline was triggered; file_validation detected the absent source, failed after its retry, and the DAG entered a failed state. The callback recorded a failed audit entry reporting zero processed records. The source filename was then restored.

Recovery confirmation

After restoring the file, the DAG was triggered again. All ten tasks succeeded and the completion task recorded 3,000 records processed. The audit history captured the following verified sequence:

Audit run ID	Status	Records processed
1	success	3,000
2	success	3,000
3	failed (intentional test)	0
4	success (recovery)	3,000

Evidence and conclusions

The results demonstrate successful multi-format parsing, reliable ordered inserts, valid customer-order references, repeatable full-refresh loading, missing-file detection, failure auditing, and recovery after the original file was restored. No automated performance benchmarks, corruption tests, or systematic database-outage tests were performed.

7. Conclusions and Future Work

Project outcome

This individual sprint delivered a functioning ten-task Airflow ETL proof of concept using Python, Faker, pandas, PyYAML, psycopg2, PostgreSQL 16, and Docker Compose. Nine source files are verified, 3,000 distinct business records are loaded in the proper dependency order, database integrity is checked, and both successful and failed DAG outcomes are recorded.

Lessons learned

The work emphasized the importance of environment configuration, predictable task dependencies, layered validation, and incremental testing. Running individual Airflow tasks before running the complete DAG helped isolate setup and import problems. Docker-based services, Airflow logs, and SQL checks supported diagnosis and verification.

Recommended improvements

Area	Suggested improvement
Deployment	Automate first-time creation of airflow_db and user setup; support a true one-command fresh deployment
Database loading	Implement atomic staging-and-swap or incremental upserts to avoid partially refreshed tables
Performance	Use parameter batching or PostgreSQL COPY for faster insertion on larger datasets
Testing	Add automated unit/integration tests and cases for corrupt files, connection failures, and schema drift
Auditing	Record actual DAG run start/end times, robust processed counts, and unique run identifiers
Validation	Check allowed categorical values, amount/date ranges, and equivalent content across formats

Project limitations and scope

This is a classroom-scale proof of concept, not a production-ready integration service. The current DAG validates three formats but inserts data from CSV only; it uses full refreshes and individual loading transactions. The audit design confirms outcome history but does not yet capture partial processing counts. These constraints are documented so future users can interpret its behavior accurately.

Submission artifacts

The Git repository contains the Airflow DAG, Faker data-generation script, SQL schema, nine original source files, Docker Compose configuration, sample environment configuration, README, and documentation. Private credentials, local virtual environments, archived copies, and database backups are excluded from Git. A live demonstration and backup video can show the DAG graph, successful run, PostgreSQL row counts, and audit history.

Summary: The implemented pipeline meets its central data-generation, orchestration, database-loading, validation, and basic error-handling objectives.